Multimodal AI with Computer Vision

SEMINAR REPORT

Submitted by

Ananda Padmanabhan PB

SNM23CA008

to

the APJ Abdul Kalam Technological University

in partial fulfillment of the requirements for the award of the Degree of

Bachelor of Technology

in

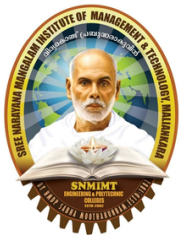
Computer Science and Engineering (Artificial Intelligence)

Under the guidance of

Ms. Surya T S

Assistant Professor

Dept. of CSE



Department of Computer Science and Engineering (Artificial Intelligence)

SREE NARAYANA MANGALAM INSTITUTE OF

MANAGEMENT AND TECHNOLOGY

MALIANKARA

October 2026

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING
(ARTIFICIAL INTELLIGENCE)

SREE NARAYANA MANGALAM INSTITUTE OF MANAGEMENT & TECHNOLOGY, MALIANKARA



CERTIFICATE

This is to certify that the seminar report entitled **Multimodal AI with Computer Vision** *submitted by* **Ananda Padmanabhan PB** *(***SNM23CA008***), to the* **Department of Computer Science and Engineering (Artificial Intelligence), SNM Institute of Management and Technology, Maliankara**, *in partial fulfilment of the requirement for the award of B.Tech Degree in Computer Science and Engineering (Artificial Intelligence) is a bonafide record of the work carried out by the student.*

Guide	Head of the Department
Ms. Surya T S	**Ms. Lakshmi Anand**
Asst. Professor	**Asst. Professor**
Dept. of CSE	**Dept. of CSE**

Coordinator

Ms. Surya T S
Asst. Professor
Dept. of CSE

Declaration

I undersigned hereby declare that the seminar report **Multimodal AI with Computer Vision**, submitted for partial fulfilment of the requirements for the award of degree of Bachelor of Technology of the APJ Abdul Kalam Technological University, Kerala is a bonafide work done by me under the supervision of **Ms. Surya T S**. This submission represents my ideas in my own words and where ideas or words of others have been included, I have adequately and accurately cited and referenced the original sources. I also declare that I have adhered to ethics of academic honesty and integrity and have not misrepresented or fabricated any data or idea or fact or source in my submission. I understand that any violation of the above will be a cause for disciplinary action by the institute and/or the University and can also evoke penal action from the sources which have thus not been properly cited or from whom proper permission has not been obtained. This report has not been previously formed the basis for the award of any degree, diploma or similar title of any other University.

Place: Maliankara

Date: 23/10/2026

Ananda Padmanabhan PB

SNM23CA008

Acknowledgement

Firstly, I thank God, the Almighty, with whose blessings, this seminar has been successfully completed.

I wish to give deep sense of acknowledgement to our Principal **Dr. Atmaram P**, for his constant encouragement and valuable advice throughout the course.

I am extremely grateful to **Ms. Lakshmi Anand**, Head of the Department of Computer Science and Engineering, for the valuable guidance through this humble endeavor. The highly enterprising attitude, patience for listening to my doubts, timely suggestions and guidance made my seminar a so-called reality in stipulated time. I would also like to thank our seminar coordinator and my seminar guide **Ms. Surya T S**, Assistant Professor, Department of Computer Science and Engineering, for the gracious encouragement and the valuable assistance.

I express my sincere gratitude to all the faculty members of the department of Computer Science and Engineering (Artificial Intelligence) for their co-operation and support to the seminar.

I also express my sincere gratitude to all my friends and my parents for their co-operation and constant inspiration.

Ananda Padmanabhan PB

Abstract

Recent advancements in Multimodal Artificial Intelligence (AI) have significantly improved the ability of intelligent systems to understand, interpret, and reason using information from multiple modalities such as images, video, and text. This seminar presents a comprehensive review of four recent research works that collectively enhance multimodal AI through latent visual reasoning, perception-aware learning, efficient computation, and video-based pretraining.

The Mirage framework introduces the concept of Machine Mental Imagery, enabling Vision-Language Models (VLMs) to perform multimodal reasoning using latent visual tokens instead of explicit image generation, thereby improving reasoning while avoiding the overhead of pixel-level synthesis. The AURORA framework enhances visual reasoning by introducing perception tokens that represent intermediate visual information such as depth maps and object locations, enabling more accurate spatial reasoning and object counting. ZipVL addresses the computational and memory challenges of large Vision-Language Models by dynamically selecting important visual tokens layer by layer, accelerating inference through adaptive token sparsity without significantly affecting accuracy. LLaVA-Prime shows that pretraining VLMs on large-scale video with causal hierarchical aggregation strengthens both temporal and static visual understanding.

Together, these research contributions demonstrate how latent reasoning, perception-aware learning, efficient computation, and richer training data can be integrated to build more accurate, efficient, and reliable multimodal intelligent systems capable of addressing complex real-world applications across computer vision, robotics, healthcare, autonomous systems, and human-computer interaction.

Keywords: Multimodal AI; Vision-Language Models; Multimodal Reasoning; Latent Visual Tokens; Perception Tokens; Dynamic Token Sparsity; Video Pretraining; Computer Vision; Deep Learning; Artificial Intelligence.

Contents

Acknowledgement **I**

Abstract **II**

List of Figures **V**

List of Tables **VI**

List of Abbreviations **VII**

1 Introduction **1**
1.1 General Background 2
1.2 Motivation 2
1.3 Problem Statement 2
1.4 Objectives 3
1.5 Scope 3
1.6 Advantages 3
1.7 Organization of the Report 3

2 Theoretical Foundations **4**
2.1 Vision Encoders and Contrastive Pretraining 4
2.2 Vision-Language Model Architectures 4
2.3 Attention, Prefill, and the Key-Value Cache 5
2.4 Multimodal Chain-of-Thought and Latent Reasoning 6
2.5 Discrete Visual Tokenisation 6
2.6 Training Paradigms 6
2.7 Benchmarks and Evaluation Metrics 7
2.8 Taxonomy of Approaches 7

3 Literature Survey **8**
3.1 Machine Mental Imagery: Empower Multimodal Reasoning with Latent Visual Tokens 8
3.1.1 Problem Statement and Motivation 8
3.1.2 Method 8
3.1.3 Experimental Setup 10
3.1.4 Results and Analysis 11
3.1.5 Discussion 12
3.2 Perception Tokens Enhance Visual Reasoning in Multimodal Language Models 12
3.2.1 Problem Statement and Motivation 12
3.2.2 Perception Tokens and Training 13

3.2.3 Experimental Setup . . . 13
3.2.4 Results and Analysis . . . 14
3.2.5 Discussion . . . 15
3.3 ZipVL: Accelerating Vision-Language Models through Dynamic Token Sparsity . . . 15
3.3.1 Problem Statement and Motivation . . . 15
3.3.2 Method . . . 16
3.3.3 Experimental Setup . . . 17
3.3.4 Results and Analysis . . . 17
3.3.5 Discussion . . . 19
3.4 Learning Beyond Still Frames: Scaling Vision-Language Models with Video 19
3.4.1 Problem Statement and Motivation . . . 19
3.4.2 Method . . . 19
3.4.3 Experimental Setup . . . 20
3.4.4 Results and Analysis . . . 21
3.4.5 Discussion . . . 22
3.5 Summary of the Reviewed Papers . . . 22

4 Discussion & Comparison **23**
4.1 Analytical Focus and Methodology . . . 23
4.2 Visual Tokens for Reasoning: Latent versus Explicit . . . 24
4.3 Efficiency and Scalability . . . 24
4.4 Training Data and Supervision . . . 24
4.5 Strengths and Limitations . . . 25
4.6 Evaluation and Overall Insights . . . 25
4.7 Towards a Combined System . . . 26

5 Applications **27**
5.1 Robotics and Embodied AI . . . 27
5.2 Autonomous Systems and Surveillance . . . 27
5.3 Healthcare and Assistive Technology . . . 27
5.4 Document, Chart, and Education Tools . . . 27

6 Challenges and Future Directions **28**
6.1 Challenges Identified in the Reviewed Works . . . 28
6.2 Cross-Cutting Challenges . . . 28
6.3 Future Research Directions . . . 28

7 Conclusion **29**

References **30**

List of Figures

1.1.1 Vision-language model pipeline and the reviewed works 2

2.3.1 Prefill and decoding phases of VLM inference 5
2.8.1 Taxonomy of the reviewed approaches 7

3.1.1 Mirage reasoning examples with latent visual tokens [1] 9
3.1.2 Two-stage training pipeline of Mirage [1] 10
3.2.1 AURORA training framework [2] 14
3.2.2 Depth maps decoded from AURORA perception tokens [2] 15
3.3.1 Attention maps across layers and tasks in an LVLM [3] 16
3.3.2 The ZipVL framework and its layer-wise token ratio [3] 17
3.3.3 Time-to-first-token versus input length (data from [3]) 18
3.4.1 The four-stage LLaVA-Prime framework [4] 20
3.4.2 Effect of scaling video pretraining data [4] 21

4.7.1 A combined pipeline using all four reviewed ideas 26

List of Tables

2.1 Benchmarks used in the reviewed works 7

3.1 Mirage on the VSP benchmark 11
3.2 AURORA on relative depth and counting 14
3.3 ZipVL on image benchmarks with LLaVA-Next-13B 18
3.4 LLaVA-Prime ablation and comparison 22
3.5 Summary of the reviewed works 22

4.1 Methodological comparison of the reviewed works 23
4.2 Strengths and limitations of the reviewed works 25

List of Abbreviations

AI	Artificial Intelligence
CHA	Causal Hierarchical Aggregation
CLIP	Contrastive Language-Image Pre-training
CoT	Chain-of-Thought
CV	Computer Vision
FLOPs	Floating-Point Operations
GPU	Graphics Processing Unit
GRPO	Group Relative Policy Optimisation
InfoNCE	Info Noise-Contrastive Estimation
KV	Key-Value (cache)
LLM	Large Language Model
LVLM	Large Vision-Language Model
MLM	Multimodal Language Model
MLP	Multi-Layer Perceptron
RL	Reinforcement Learning
SFT	Supervised Fine-Tuning
t-SNE	t-Distributed Stochastic Neighbour Embedding
TTFT	Time-to-First-Token
ViT	Vision Transformer
VLM	Vision-Language Model
VQ-VAE	Vector-Quantised Variational Autoencoder
VQA	Visual Question Answering
VSP	Visual-Spatial Planning

Chapter 1

Introduction

For most of its history, computer vision built separate models for separate tasks: one network classified images, another detected objects, and a third estimated depth. **Multimodal Artificial Intelligence** (AI) changes this picture by learning from, and reasoning across, several kinds of data at once—images, video, and natural language. Its central building block is the **Vision-Language Model** (VLM), which encodes an image into a sequence of visual tokens and passes them, together with text, to a large language model (LLM) that can answer questions, describe scenes, and follow instructions [6, 5].

Today's VLMs are impressive generalists, but they reason almost entirely in *words*, which suits description but not spatial planning [1]; they are weak at *perception* such as judging depth or counting [2]; they are *expensive*, because high-resolution images and videos produce thousands of visual tokens [3]; and their image-text training data teaches little about how scenes change over time [4].

This seminar reviews four recent works from CVPR and ICCV that address these limitations. Paper [1], the base paper, by Yang et al., introduces **Mirage**, which lets a VLM "think visually" with latent visual tokens instead of generating images. Paper [2], by Bigverdi et al., introduces **AURORA**, which teaches a VLM to produce *perception tokens* such as depth maps and bounding boxes as intermediate reasoning steps. Paper [3], by He et al., presents **ZipVL**, which accelerates VLM inference through dynamic token sparsity. Paper [4], by Zhang et al., presents **LLaVA-Prime**, which scales VLM pretraining with video.

1.1 General Background

Figure 1.1.1 shows the typical VLM pipeline that all four works build on. A **vision encoder**, usually a Vision Transformer (ViT) [7] pretrained with CLIP-style contrastive learning [5], splits the image into patches and turns them into feature vectors. A small **projector**, often a multi-layer perceptron (MLP), maps these features into the LLM's embedding space, producing **visual tokens**. The LLM then processes visual and text tokens together and generates the answer one token at a time [6]. Each of the reviewed works intervenes at a different point in this pipeline: [4] at pretraining of the encoder and LLM, [3] at the attention computation inside the LLM, and [1, 2] at the output, by letting the model emit new kinds of tokens while it reasons.

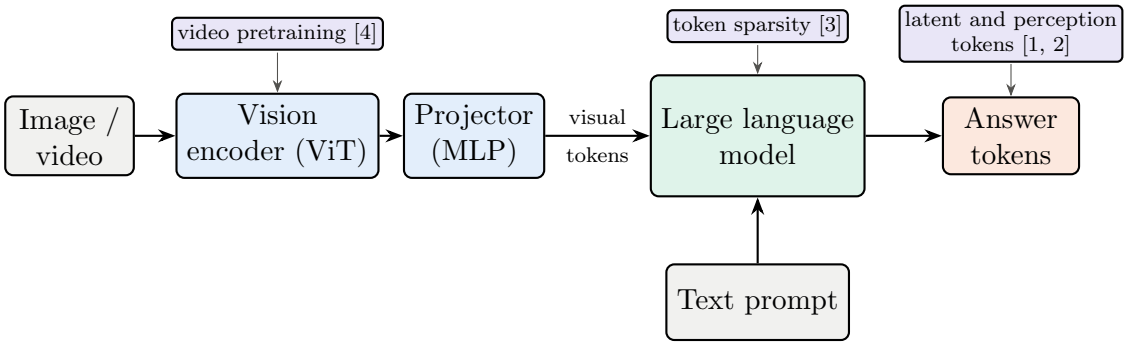


Figure 1.1.1: Vision-language model pipeline and the reviewed works

1.2 Motivation

Three needs motivate this study: **deeper visual reasoning**, since navigation, assembly, and spatial questions require manipulating visual information rather than describing it [1, 2]; **efficiency**, since high-resolution images, long videos, and edge hardware demand less computation and memory [3]; and **better data**, since image-text pairs are costly and lack temporal information [4].

1.3 Problem Statement

Given a vision-language model that encodes images or videos into tokens and reasons in text, enable it to reason with visual information directly, to perceive depth and object instances reliably, to run efficiently on long visual inputs, and to learn temporal dynamics—while keeping a single, general-purpose architecture.

1.4 Objectives

The objectives of this seminar are:

- To introduce vision encoders, visual tokens, VLM architectures, attention and key-value caching, and multimodal chain-of-thought reasoning.
- To review in depth four recent works on latent visual reasoning, perception tokens, token sparsity, and video pretraining.
- To compare their methods, training strategies, and evaluation, and to identify open challenges.

1.5 Scope

The seminar covers image and video understanding with transformer-based VLMs, including visual question answering, spatial reasoning, depth estimation, counting, and efficient inference, together with the benchmarks used to evaluate them. Image and video *generation* models are discussed only where they relate to reasoning.

1.6 Advantages

Multimodal AI with computer vision offers several advantages over task-specific vision systems:

- **Generality:** one model handles many tasks through natural-language instructions.
- **Richer Reasoning:** visual and textual evidence are combined in a single reasoning process.
- **Natural Interaction:** users ask questions about images and videos in plain language.
- **Transferability:** knowledge from web-scale data transfers to new tasks with little extra training.

1.7 Organization of the Report

Chapter 2 introduces the theoretical foundations. **Chapter 3** reviews the four works in detail. **Chapter 4** compares the approaches, **Chapter 5** discusses applications, **Chapter 6** presents challenges and future directions, and **Chapter 7** concludes the report.

Chapter 2

Theoretical Foundations

This chapter introduces the concepts on which the reviewed works build: vision encoders and contrastive pretraining, VLM architectures, attention and the key-value cache, multimodal chain-of-thought and latent reasoning, discrete visual tokenisation, training paradigms, and evaluation benchmarks.

2.1 Vision Encoders and Contrastive Pretraining

A **Vision Transformer** (ViT) [7] splits an image into fixed-size patches, flattens each patch into a vector, adds a position embedding, and processes the resulting sequence with transformer layers, so that every patch becomes a feature vector informed by the whole image. Vision encoders for VLMs are usually pretrained with **CLIP** [5], which trains an image encoder and a text encoder together on image-caption pairs so that matching pairs have similar embeddings. For a batch of N_B pairs with image embeddings $z_v^{(i)}$ and text embeddings $z_t^{(i)}$, the **InfoNCE** loss is

$$\mathcal{L}_{\text{CLIP}} = -\frac{1}{N_B}\sum_{i=1}^{N_B}\log\frac{\exp\left(\text{sim}(z_v^{(i)}, z_t^{(i)})/\tau\right)}{\sum_{j=1}^{N_B}\exp\left(\text{sim}(z_v^{(i)}, z_t^{(j)})/\tau\right)}, \tag{2.1}$$

where sim is cosine similarity and τ a temperature. Because captions describe what is in an image, CLIP encoders learn general visual concepts and can recognise new categories zero-shot [4].

2.2 Vision-Language Model Architectures

Multimodal language models (MLMs) combine a vision encoder with an LLM in two main ways. **Cross-attention** architectures encode the image separately and let the language model attend to its features; **visual instruction tuning**, popularised by LLaVA

[6], instead projects image features into the LLM's embedding space and interleaves them with text tokens [2]. The second design dominates current work, including all four reviewed papers. Given a sequence $s = (s_1, \ldots, s_T)$ of visual and text tokens, the model is trained with the **next-token prediction** loss

$$\mathcal{L}_{\text{NTP}} = -\sum_{i=1}^{T-1} \log p_\theta\big(s_{i+1} \mid s_1, \ldots, s_i\big), \tag{2.2}$$

usually computed only on the answer tokens during instruction tuning [4]. The LLaVA and Qwen-VL families are the base models used throughout the reviewed works.

2.3 Attention, Prefill, and the Key-Value Cache

Inside the LLM, each transformer layer computes **self-attention** [8]. For input $X \in \mathbb{R}^{n \times d}$ with n tokens, queries, keys, and values are $Q = XW_Q$, $K = XW_K$, and $V = XW_V$, and

$$A = \text{Softmax}\left(\frac{QK^\top + M}{\sqrt{d}}\right), \qquad O = AV, \tag{2.3}$$

where M is a causal mask that prevents tokens from attending to later ones [3]. Generation has two phases (Figure 2.3.1). In the **prefill** phase, all prompt tokens—including thousands of visual tokens—are processed at once; computing $QK^\top$ costs $O(n^2)$, so this phase is *compute-bound* and determines the time-to-first-token (TTFT). In the **decoding** phase, one new token is produced per step; to avoid recomputation, the keys and values of all previous tokens are stored in a **key-value (KV) cache**, which must be read from memory at every step, so this phase is *memory-bound* [3].

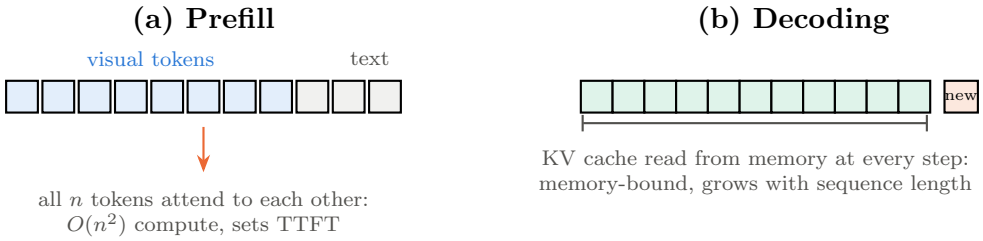


Figure 2.3.1: Prefill and decoding phases of VLM inference

2.4 Multimodal Chain-of-Thought and Latent Reasoning

Chain-of-thought (CoT) prompting [9] elicits step-by-step reasoning by having a model write out intermediate steps before its answer. Multimodal extensions interleave visual evidence such as image crops, boxes, or tool outputs with these steps, and unified models such as Chameleon and Anole generate interleaved text and images at the cost of pixel-level pretraining [1]. LLMs can also reason in **latent space**, replacing some reasoning tokens with continuous hidden-state vectors. The reviewed works combine these ideas: [2] adds discrete perception tokens as intermediate steps, and [1] adds continuous latent visual tokens.

2.5 Discrete Visual Tokenisation

A **Vector-Quantised Variational Autoencoder** (VQ-VAE) [10] represents an image, or an intrinsic map such as depth, as a grid of discrete codes. An encoder maps the input to a grid of vectors z_e, each vector is replaced by its nearest entry in a learned codebook $\{e_k\}_{k=1}^{K}$,

$$z_q = e_{k^*}, \qquad k^* = \arg\min_k \left\| z_e - e_k \right\|_2, \tag{2.4}$$

and a decoder reconstructs the input from the codes. The code indices form a short token sequence that a language model can predict like words, which is how [2] turns depth maps into perception tokens.

2.6 Training Paradigms

Besides **supervised fine-tuning** (SFT) with the next-token loss, the reviewed works use **curriculum learning**, which orders tasks by difficulty to avoid catastrophic forgetting [2]. **Distillation** trains a model to match targets produced by another model or encoder, such as image embeddings [1] or specialist depth predictions [2, 4]. Finally, **reinforcement learning** with **Group Relative Policy Optimisation** (GRPO) [11] samples several responses per question, rewards correct and well-formatted answers, and favours responses that score above the group average, without a separate value network [1].

2.7 Benchmarks and Evaluation Metrics

The reviewed works evaluate on a wide range of benchmarks (Table 2.1). Accuracy is the main metric for question answering, reasoning, and counting, with CIDEr for captioning and recall at k for retrieval. Efficiency is measured by time-to-first-token, decoding throughput in tokens per second, the share of tokens kept, the reduction in attention floating-point operations (FLOPs), and the reduction in KV-cache size [3].

Table 2.1: Benchmarks used in the reviewed works

Category	**Benchmarks**	**Used in**
Spatial reasoning	VSP spatial reasoning and planning, BLINK-Jigsaw, SAT, COMT geometry	[1]
Depth and counting	BLINK relative depth, HardBLINK, CV-Bench, SEED-Bench, BLINK counting	[2]
Image understanding	VQAv2, GQA, TextVQA, ChartQA, MME, MMBench, MMVet, SEED	[3, 4]
Video understanding	Video-MME, EgoSchema, MSVD, ActivityNet, MLVU	[3, 4]
Encoder evaluation	Zero-shot classification, Kinetics, Flickr30K and COCO retrieval	[4]

2.8 Taxonomy of Approaches

Figure 2.8.1 organises the reviewed works by the limitation of VLMs they address. Two works enrich *reasoning* with new kinds of visual tokens, one improves *efficiency*, and one improves the *data and pretraining* from which visual understanding is learned.

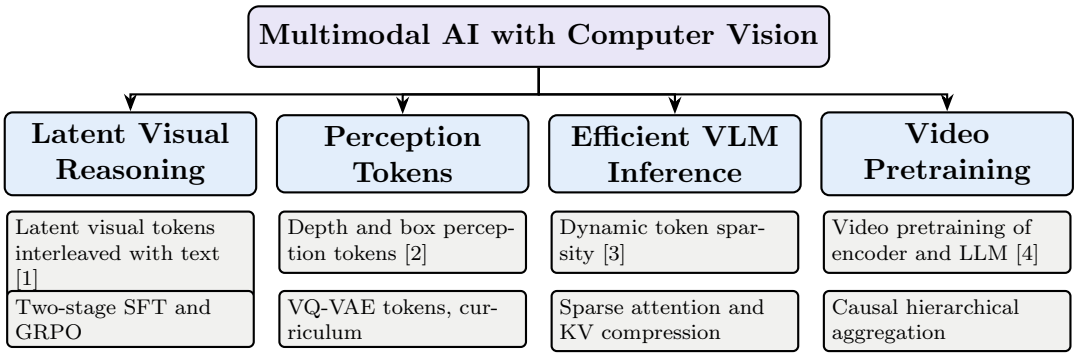


Figure 2.8.1: Taxonomy of the reviewed approaches

Chapter 3

Literature Survey

This chapter reviews the four works in detail. For each, it describes the problem and motivation, the method, the experimental setup and results, and closes with a short discussion of strengths and limitations.

3.1 Machine Mental Imagery: Empower Multimodal Reasoning with Latent Visual Tokens

The base paper, by Yang, Yu, Chen, Shen, and Gan [1], from the University of Massachusetts Amherst, MIT, and the MIT-IBM Watson AI Lab, introduces **Mirage**, a framework that lets a VLM reason with compact latent visual tokens interleaved with ordinary text.

3.1.1 Problem Statement and Motivation

VLMs decode only text, so they must *verbalise* visual reasoning. This limits them on tasks such as spatial planning and jigsaw puzzles, where people instead picture how pieces fit or how a path unfolds. Recent unified models can generate images while reasoning, but heavy image-generation pretraining tends to degrade reasoning, and their image decoders cannot produce interleaved trajectories tied to the input images [1]. Imagery theory suggests that humans do not imagine photorealistic pictures; they build simplified *mental images* that keep only task-relevant cues. Mirage asks whether a VLM can do the same by reasoning directly in its latent visual embedding space, without producing pixels (Figure 3.1.1).

3.1.2 Method

Data synthesis. Each training question x with answer y is paired with a task-specific *helper image* I that contains exactly the visual cue needed: for maze navigation, the

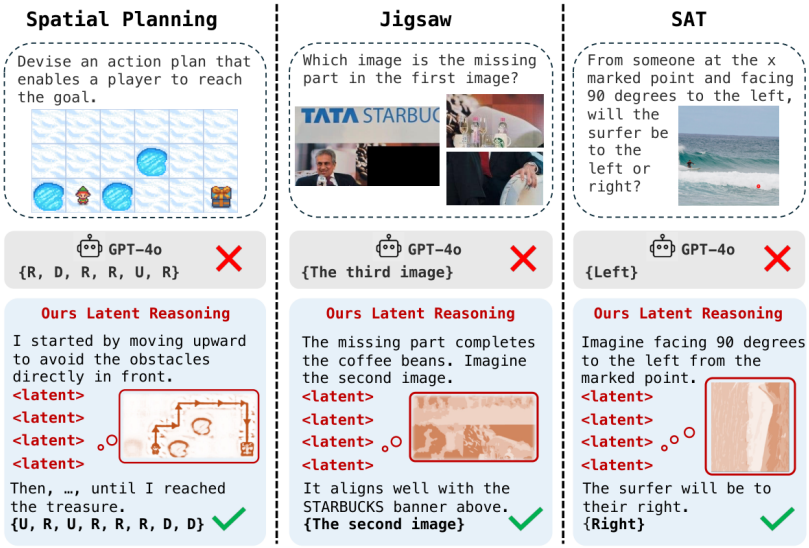


Figure 3.1.1: Mirage reasoning examples with latent visual tokens [1]

start map with the correct path drawn in red arrows; for jigsaw, a candidate piece placed beside the reference image; for spatial-aptitude questions, a scene rendered by a fine-tuned CogVideoX-5B video model. A large reasoning VLM, Qwen2.5-VL-32B, is then prompted to write a step-by-step solution that incorporates the helper image, producing a trajectory $o = o_{\text{pre}} \oplus I \oplus o_{\text{post}}$ in which the image splits the text into a part before and a part after it [1].

Stage 1: joint supervision. Rather than asking the model to generate the helper image, Mirage passes it through the VLM to obtain patch features $\{e_1, \ldots, e_n\}$ and compresses them by average pooling into k salient vectors $\{\hat{e}_1, \ldots, \hat{e}_k\}$. The model is trained to produce o_{pre}, then to output these k vectors as its last-layer hidden states at the latent positions, and then to produce o_{post}. The latent tokens are supervised with a cosine loss,

$$\mathcal{L}_{\text{visual}} = \sum_{j=1}^{k} \ell_{\cos}\big(\hat{e}_j,\ g_\theta(o_{\text{pre}}, \hat{e}_{1:j-1})\big), \tag{3.1}$$

and the text with the usual cross-entropy loss $\mathcal{L}_{\text{text}}$, giving $\mathcal{L}_1 = \mathcal{L}_{\text{visual}} + \gamma\,\mathcal{L}_{\text{text}}$. This anchors the latent tokens in the visual subspace while teaching the model to weave them into its thoughts (Figure 3.1.2).

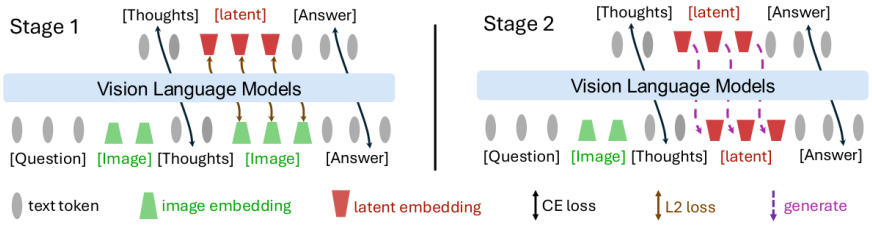


Figure 3.1.2: Two-stage training pipeline of Mirage [1]

Stage 2: text-only supervision. Forcing latent tokens to reconstruct image embeddings can over-constrain the model and divert capacity from answering correctly. In the second stage, the cosine loss is removed: the model generates its own latent tokens autoregressively, $e_j = f_\theta(x, o_{\text{pre}}, e_{<j})$, and only the text tokens are supervised. Because the latent tokens are continuous, gradients from the text loss flow back through them, so they adapt freely within the learned visual subspace and act as priors for the text that follows [1].

Reinforcement learning. Finally, the model is refined with GRPO [11], sampling several responses per question. The reward is 1 for a correct final answer and an extra 0.1 when the reasoning is enclosed in think tags and the answer is given in the required boxed format; gradients again flow through the latent tokens [1].

3.1.3 Experimental Setup

The default base model is Qwen2.5-VL 7B with $k = 4$ latent tokens and loss coefficient $\gamma = 0.1$. Four benchmarks are used: **VSP**, with spatial-reasoning and spatial-planning tasks in a simulated maze; **BLINK-Jigsaw**; **SAT**, with static and dynamic spatial relations; and the geometry subset of **COMT**. For each task, 1k examples are used for fine-tuning and 1k for reinforcement learning. Baselines include zero-shot and directly fine-tuned models, CoT fine-tuning with and without GRPO, unified models that generate images (Anole and MVoT), and interleaved reasoning models such as AURORA [2], ViGoRL, MindJourney, and MINT-CoT [1].

3.1.4 Results and Analysis

Table 3.1 shows average accuracy on the two VSP tasks. Latent visual tokens improve both reasoning styles: Mirage beats direct fine-tuning by 3 points on spatial reasoning and CoT fine-tuning by 11 points on spatial planning, and GRPO adds a further 2 points. Models that explicitly generate images perform poorly; even after fine-tuning on the same data, AURORA reaches 0.71 and 0.13, which the authors attribute to the overhead of image generation [1]. With the smaller Qwen2.5-VL 3B, Mirage reaches 0.85 on Jigsaw, 0.89 on synthetic SAT, and 0.64 on real SAT, against 0.80, 0.83, and 0.55 for direct fine-tuning, and on COMT geometry it scores 0.77 against 0.72 for MINT-CoT, which was trained on large-scale mathematics data.

Table 3.1: Mirage on the VSP benchmark

Method (Qwen2.5-VL 7B)	**Spatial reasoning (avg.)**	**Spatial planning (avg.)**
Zero-shot	0.32	0.06
Direct SFT	0.83	0.72
CoT SFT	0.84	0.47
CoT SFT + GRPO	0.85	0.51
Anole / MVoT / AURORA	0.52 / 0.61 / 0.71	0.01 / 0.11 / 0.13
Mirage (direct)	0.86	**0.76**
Mirage (CoT)	0.87	0.58
Mirage (CoT) + GRPO	**0.89**	0.60

Ablations. Removing Stage 1 lowers spatial-planning accuracy from 0.58 to 0.52, because ungrounded latent vectors drift to unhelpful regions of the embedding space; removing Stage 2 leaves the latent tokens over-constrained and drops it to 0.21, so both stages are necessary. Performance is stable for k between 2 and 6 latent tokens but falls by about 13 points at $k = 8$, consistent with error accumulation over longer latent sequences, and every tested value of γ still exceeds 80% accuracy after Stage 2 [1].

Latent behaviour. Supplying the helper images directly as input lets models reach nearly 100% on the VSP tasks, confirming that the synthesised cues are informative and indicating an upper bound for latent reasoning. A t-SNE projection shows that Mirage's latent tokens form a compact cloud just outside the cluster of visual embeddings and far

from the text embeddings: they remain visual in character but are shifted by Stage 2 towards what helps answer generation, echoing how mental imagery abstracts rather than reproduces what is seen [1].

3.1.5 Discussion

Mirage's main strength is that it adds visual thinking to a VLM cheaply: four latent vectors replace a generated image, no image decoder is needed, and the approach transfers from 7B to 3B models. Its two-stage design is well supported by ablations. Its limitations are the reliance on synthesised helper images and reasoning traces, which can be noisy—especially for SAT, whose helper images come from a video generator without ground truth—and evaluation on four spatial benchmarks with 1k training examples each. On VSP planning, fine-tuning on reasoning traces also underperforms fine-tuning on answers alone, showing that explicit reasoning does not always help perception-heavy tasks.

3.2 Perception Tokens Enhance Visual Reasoning in Multimodal Language Models

Bigverdi, Luo, Hsieh, Shen, Chen, Shapiro, and Krishna [2], from the University of Washington and Google Research, introduce **perception tokens** and **AURORA**, a training method that teaches multimodal language models to produce and reason over them.

3.2.1 Problem Statement and Motivation

MLMs still fail at basic perception tasks where specialist vision models excel. Deciding which of several marked points is closest to the camera is hard to reason about in words but easy with a depth map, and counting objects is easier with their bounding boxes. Fine-tuning MLMs on such tasks generalises poorly, and calling external depth estimators or detectors is compute- and memory-intensive and fragile [2]. The authors therefore let the model generate intrinsic image representations itself, as intermediate reasoning steps akin to chain-of-thought [9]: for example, "The depth map is ⟨perception tokens⟩. Therefore, point D is closest to the camera."

3.2.2 Perception Tokens and Training

The model's vocabulary V is expanded to $V' = V \cup V_{\text{aux}}$ with auxiliary perception tokens. Two kinds are used. **Pixel-level tokens** encode depth maps: a VQ-VAE [10] with a 128-entry codebook compresses each 320×320 depth map into a 10×10 grid, giving 100 tokens drawn from 128 depth tokens plus start and end markers, 130 new tokens in all; it is trained on pseudo-depth maps of ADE20k images produced by Depth Anything. **Structured tokens** encode bounding boxes: images are resized to 336×336 and 336 coordinate tokens are added, so a box is written as four tokens (x_1, y_1, x_2, y_2) [2].

To learn to *predict* perception tokens, AURORA distils from specialist models: if a specialist gives a target distribution q_i over its tokens and $\mathcal{M}$ maps them to auxiliary tokens, the loss is

$$\ell_{\text{dist}} = -\sum_i q_i \log p_{\mathcal{M}(i)}, \tag{3.2}$$

optionally combined with a reconstruction loss $\ell_{\text{rec}} = \|g(T) - f\|_2^2$ through a lightweight decoder g that maps a token set T back to its target feature f. To learn to *reason* with them, training mixes three subsets of increasing difficulty—generating perception tokens, answering with chain-of-thought through perception tokens, and answering directly—and samples task t of difficulty d_t at step s with a curriculum

$$p(d_t, s) = \frac{\exp\big(-d_t/\tau(s)\big)}{\sum_{i=1}^{m} \exp\big(-d_i/\tau(s)\big)}, \qquad \tau(s) = \frac{\tau_0}{1 + \lambda \cdot s/S}, \tag{3.3}$$

which shifts sampling between tasks over training to avoid catastrophic forgetting (Figure 3.2.1). At inference, constrained decoding restricts sampling to auxiliary tokens at the steps where they are expected [2].

3.2.3 Experimental Setup

AURORA is applied to LLaVA 1.5 13B, giving **LLaVA-AURORA**. For relative depth, training uses 20k ADE20k depth-generation examples and 500 images with 2–5 markers for chain-of-thought and direct answers; evaluation uses BLINK relative depth and **Hard-BLINK**, harder variants with 3, 4, and 5 markers placed at mid-height to remove height cues. For counting, training uses 5k LVIS images with 0–15 objects and 250 images for

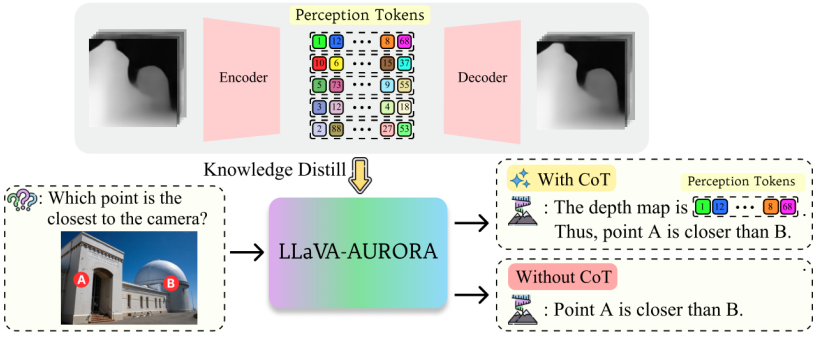


Figure 3.2.1: AURORA training framework [2]

reasoning; evaluation uses CV-Bench, SEED-Bench, and BLINK counting, all without multiple-choice options. Baselines include LLaVA fine-tuned on the same direct-answer data, LLaVA-OneVision, GPT-4o, GPT-4 Turbo, and GPT-4 Turbo with tools [2].

3.2.4 Results and Analysis

Table 3.2 summarises the results. On relative depth, LLaVA-AURORA averages 61.6% over BLINK and HardBLINK, against 55.2% for the fine-tuned baseline, and its advantage grows with the number of points, reaching 54.8% on five points against 41.1%. It also surpasses GPT-4o and GPT-4 Turbo, even when the latter is given ground-truth depth maps. On counting, it improves on the fine-tuned baseline by 11.3 points on CV-Bench and 8.3 on SEED-Bench, and on the base model by 10.8 on BLINK, where the fine-tuned baseline collapses [2]. Decoded depth maps resemble the ground truth (Figure 3.2.2), and their programmatic relative-depth accuracy, 79.2% on average, exceeds Unified-IO's 74.4%.

Table 3.2: AURORA on relative depth and counting

Model	Depth avg.	Depth 5-pt	CV-Bench	SEED	BLINK
LLaVA 1.5 13B	39.1	29.0	40.9	52.2	35.0
Fine-tuned LLaVA	55.2	41.1	44.7	46.3	0.2
LLaVA-OneVision	53.4	44.3	34.4	31.7	35.8
GPT-4o	49.6	36.3	70.2	64.6	47.5
GPT-4 Turbo + Tool	49.6	26.6	48.6	29.9	26.7
LLaVA-AURORA	**61.6**	**54.8**	56.0	54.6	45.8

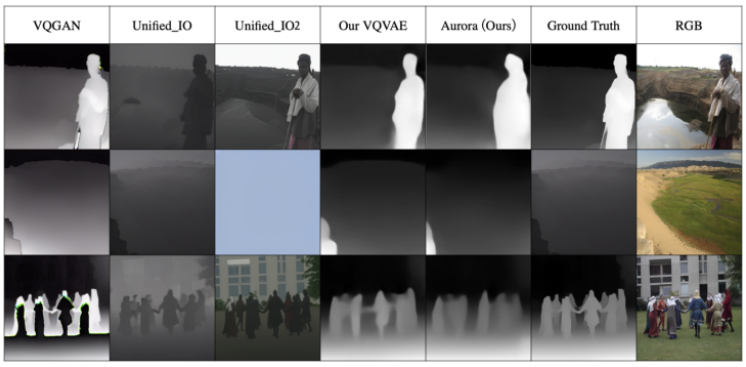


Figure 3.2.2: Depth maps decoded from AURORA perception tokens [2]

3.2.5 Discussion

AURORA shows that a single MLM can learn to produce useful intermediate perception without external tools, and that chain-of-thought over such tokens generalises across benchmarks better than plain fine-tuning. Its framework is extensible to other intrinsics such as segmentation masks or poses. Its limitations are the narrow evaluation, two tasks on one base model, a slightly lower score than the fine-tuned baseline on the original two-point BLINK depth task (64.5% against 68.5%), and counting accuracy still below closed-source models such as GPT-4o. Mirage [1] later found that explicit token generation of this kind is less effective for interleaved spatial reasoning than compact latent tokens.

3.3 ZipVL: Accelerating Vision-Language Models through Dynamic Token Sparsity

He, Chen, Liu, Shao, Zhou, Zhang, and Zhuang [3], from Zhejiang University, the Shanghai AI Laboratory, the University of Adelaide, and Monash University, present **ZipVL**, an inference framework that speeds up both phases of large VLM (LVLM) generation with a single, dynamically chosen set of important tokens.

3.3.1 Problem Statement and Motivation

High-resolution images and videos produce very long token sequences—a 128-frame video becomes over 18,000 tokens in the LongVA model. The prefill phase then suffers from quadratic attention cost and a long time-to-first-token, and the decoding phase from repeatedly reading a large KV cache [3]. Visual content is highly redundant, so attention

maps in LVLMs are sparse, but their sparsity varies across layers and tasks (Figure 3.3.1). Existing methods tackle only one bottleneck, rely on predefined sparse patterns that need custom GPU kernels, or keep a fixed share of tokens—FastV, for example, drops half of the visual tokens after the second layer and degrades on hard tasks such as ChartQA.

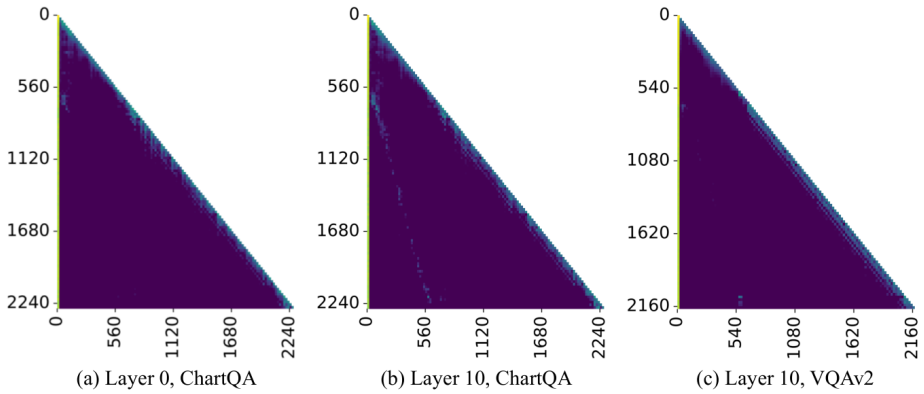


Figure 3.3.1: Attention maps across layers and tasks in an LVLM [3]

3.3.2 Method

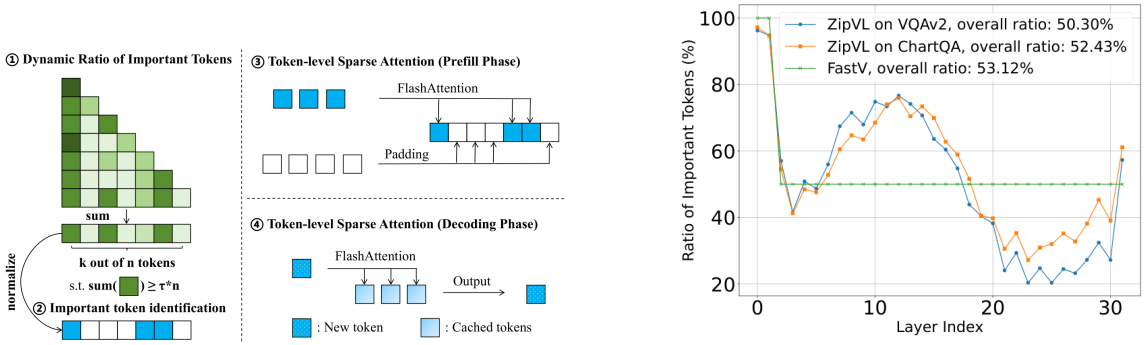
Layer-wise adaptive ratio. In each layer, ZipVL sums the attention each token receives, $a_j = \sum_{c=1}^{n} A_{c,j}$, sorts these scores, and keeps the smallest number of tokens that preserves a fraction τ of the total attention:

$$p = \min \left\{ p \in \mathbb{Z} \;\middle|\; \sum_{j=1}^{p} a_{\text{sorted}(j)} \geq \tau \times n \right\}, \tag{3.4}$$

where the total is n because every row of A sums to one. Simple tasks concentrate attention on few tokens and get a small p; harder tasks spread attention and get a larger p, so the ratio adapts to each layer and task without hand-set hyperparameters (Figure 3.3.2b) [3].

Important-token selection. Tokens are ranked by their *normalised* attention score, $\tilde{a}_j = \sum_c A_{c,j} / \text{nnz}(A_{:,j})$, which corrects for the causal mask giving early tokens more attention, and the top p form the important set T. In the **prefill** phase, attention is computed only among the important tokens using standard FlashAttention, and the outputs of the other tokens are padded; in the **decoding** phase, only the important tokens' keys and

values are kept in the cache. Because FlashAttention never materialises the full attention matrix, the scores are approximated from a small, fixed set of probe tokens (Figure 3.3.2a, Algorithm 1) [3].



(a) Framework overview **(b)** Important-token ratio per layer

Figure 3.3.2: The ZipVL framework and its layer-wise token ratio [3]

Algorithm 1 Prefill phase of ZipVL (adapted from [3])

Require: input embeddings X; attention-retention threshold τ

1: Compute Q, K, V from X
2: Compute attention scores A' for a small set of probe queries against all keys
3: Find the number of important tokens p from the sorted accumulated scores
4: Compute normalised scores $\tilde{a}_j$ and select $T = \text{topk}(\tilde{a}, p)$
5: $O \leftarrow \text{FlashAttention}(Q[T], K[T], V[T])$; pad outputs of the remaining tokens
6: Keep only $K[T]$ and $V[T]$ in the KV cache for decoding

3.3.3 Experimental Setup

Image understanding is evaluated with LLaVA-1.5-7B, LLaVA-Next-7B, and LLaVA-Next-13B on VQAv2, TextVQA, GQA, MME, and ChartQA, against FastV and HiRED. Video understanding is evaluated with LongVA-7B on Video-MME with 64 and 128 frames, against MInference, QK-sparse, and FastV. Latency and throughput are measured for LLaVA-Next-13B on an NVIDIA A100 GPU [3].

3.3.4 Results and Analysis

Accuracy. Table 3.3 shows results on LLaVA-Next-13B. With $\tau = 0.975$, ZipVL keeps only 36.8% of tokens yet stays within 0.5 points of the full model on VQAv2 and matches it on MME, while FastV, keeping 53.1%, loses much more, especially on ChartQA. ZipVL outperforms FastV on nearly all benchmarks across the three models, and HiRED, which

ignores the text prompt when choosing patches, loses accuracy, especially at low token ratios. On Video-MME with 128 frames, ZipVL matches the full model's 52.6% while cutting attention FLOPs by 82.3% and the KV cache by 57.9%; a fixed ratio with the same overall sparsity reaches only 51.1% [3].

Table 3.3: ZipVL on image benchmarks with LLaVA-Next-13B

Method	Tokens kept	VQAv2	ChartQA	TextVQA	GQA	MME
Full attention	100%	80.9	66.2	66.9	65.7	1570
FastV	53.1%	76.8	51.6	59.7	62.9	1555
HiRED	40%	79.3	53.7	65.2	64.1	1570
ZipVL ($\tau = 0.96$)	30.6%	79.7	57.8	63.6	64.3	1551
ZipVL ($\tau = 0.975$)	36.8%	80.4	58.5	64.9	64.8	1574

Efficiency. Figure 3.3.3 plots the time-to-first-token. For short inputs ZipVL matches FlashAttention, but as inputs grow the attention module dominates and the savings rise, reaching a 2.3 times reduction at 128K tokens (20.57 s against 48.01 s). MInference is slower than FlashAttention below 32K tokens, and FastV, which needs full attention scores, runs out of memory beyond 8K. Because the smaller KV cache allows larger batches, ZipVL also raises decoding throughput by about 2.8 times at 16K tokens (40.87 against 14.10 tokens/s) and keeps decoding at 32K and 64K, where the baselines run out of memory. Applied to KV-cache quantisation of LLaMA3-8B, the adaptive ratio also gives a higher compression ratio than ZipCache (6.18 against 4.69 times) with better accuracy [3].

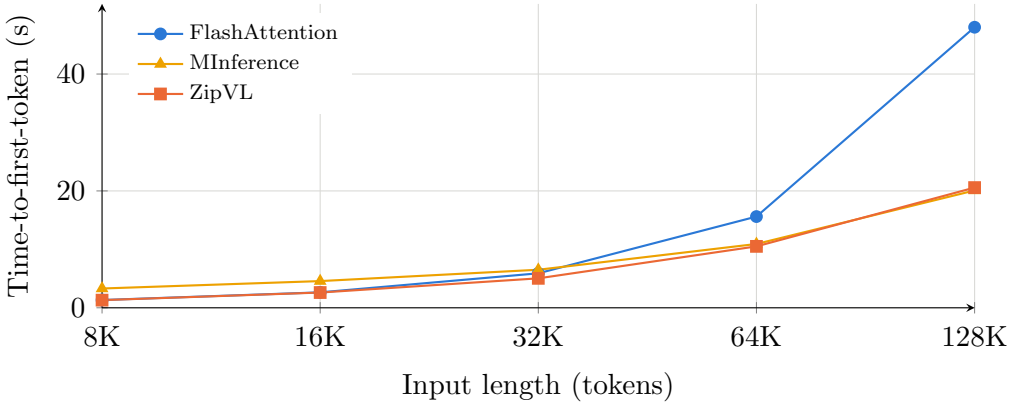


Figure 3.3.3: Time-to-first-token versus input length (data from [3])

3.3.5 Discussion

ZipVL's strengths are that one signal—the attention distribution—drives both sparse prefill and KV-cache eviction, that it works with standard FlashAttention without custom kernels, and that its ratio adapts automatically to layers, tasks, and models. Its limitations are that only attention is sparsified while the MLP layers stay dense, that probe-based score approximation adds overhead at short lengths, and that hard tasks such as ChartQA still lose several points; the speed-ups appear mainly for long inputs.

3.4 Learning Beyond Still Frames: Scaling Vision-Language Models with Video

Zhang, Li, Liu, and Yue [4], from CUHK, UCAS, CASIA, and the Shanghai AI Laboratory, propose **LLaVA-Prime**, which adds large-scale video pretraining to the LLaVA recipe to improve both image and video understanding.

3.4.1 Problem Statement and Motivation

VLMs depend on high-quality image-text pairs, which are expensive to create, and even good image data captures isolated moments with no temporal context, limiting understanding of motion, events, and causality in video. Videos offer richer visual information, because consecutive frames show how scenes evolve, and their captions are longer and more detailed, which encourages finer vision-language alignment. Video, however, is costly to process and requires strict temporal causality [4].

3.4.2 Method

Starting from LLaVA-1.5 with a 1-billion-parameter ViT-g vision encoder and the Qwen2-7B LLM, LLaVA-Prime trains in four stages (Figure 3.4.1). **Stage I** pretrains the vision encoder on video-caption pairs from Panda-70M with the contrastive InfoNCE loss, feeding four key frames reshaped into a single 12-channel input. **Stage II** pretrains the LLM on interleaved video and text tokens with the next-token loss, stabilised by EVE's patch-aligning layer, and distils the Stage I encoder's features into the LLM's

vision tokens with a token-wise squared error,

$$\mathcal{L}_{\text{distill}} = \frac{1}{N_v} \sum_{i=1}^{N_v} \left\| z_{v,i} - e_{v,i} \right\|_2^2, \tag{3.5}$$

where z_v and e_v are the encoder and LLM representations. **Stage III** trains on joint image-video-text data with the new aggregation module, and **Stage IV** performs visual instruction tuning with the encoder frozen [4].

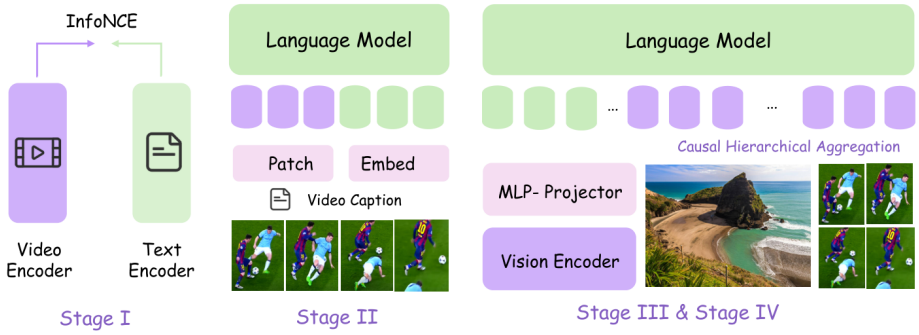


Figure 3.4.1: The four-stage LLaVA-Prime framework [4]

Causal Hierarchical Aggregation (CHA) lets the same image encoder process video while keeping the number of video tokens equal to that of an image. Following run-length tokenisation, a video is represented by its first frame x_v^0 and the differences between consecutive frames, $\Delta x_v^{i-1,i} = x_v^i - x_v^{i-1}$. A causal 1D convolution aggregates these over time so that each step depends only on the past,

$$z_v^0 = w \cdot x_v^0, \qquad z_v^i = z_v^{i-1} \cdot M_i \cdot w \cdot \Delta x_v^{i-1,i}, \tag{3.6}$$

and several dilated, large-kernel causal convolutions—dilations 1, 3, 5, and 7 with kernel sizes 13, 13, 9, and 7—build hierarchical temporal receptive fields, compressing $N_v \times T$ video tokens to N_v [4].

3.4.3 Experimental Setup

LLaVA-Prime-7B is trained like LLaVA-OneVision-7B plus 32M Panda-70M video samples (about 25B video tokens at one frame per second), and LLaVA-Prime-72B with 128M

samples (about 100B video tokens), on 512 NVIDIA A100 GPUs for two weeks. The encoder is evaluated on zero-shot classification over 27 datasets, Kinetics video recognition, and Flickr30K and COCO retrieval; the full model on image benchmarks such as GQA, VQAv2, MME, MMBench, MMVet, and SEED, and on video benchmarks such as MSVD, ActivityNet, EgoSchema, Video-MME, and MLVU [4].

3.4.4 Results and Analysis

Scaling. Increasing video pretraining data from 2M to 128M samples steadily improves image and video understanding (Figure 3.4.2): general image benchmarks rise from 66.2 to 72.2, image captioning from 33.1 to 43.8, and Video-MME from 56.2 to 63.8. Longer frame sequences and context lengths also help, and each training stage contributes in the ablation (Table 3.4) [4].

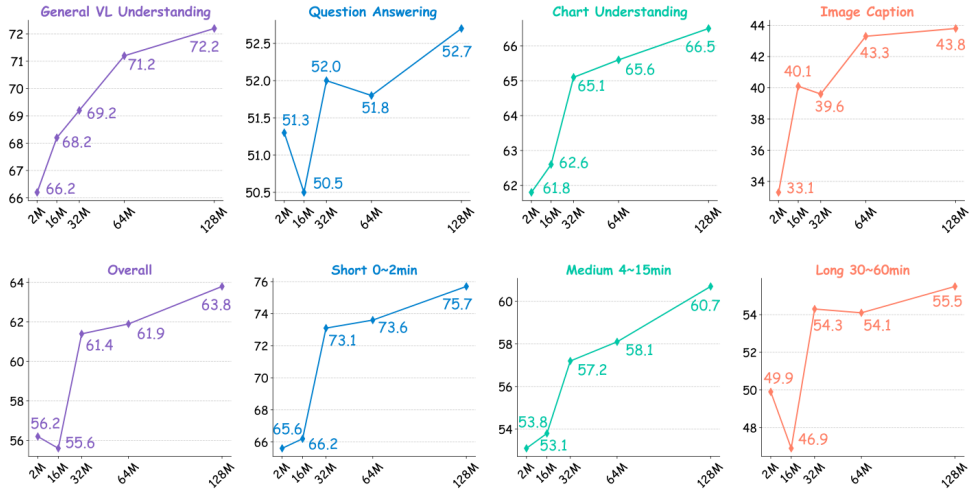


Figure 3.4.2: Effect of scaling video pretraining data [4]

Comparison. The video-pretrained 1B encoder averages 78.5% zero-shot accuracy across 27 classification datasets, against 72.4% for the EVA-01-CLIP-g/14 it starts from, outperforming the 6B InternVL-C encoder. LLaVA-Prime-7B surpasses LLaVA-OneVision-7B on both image and video benchmarks, and video pretraining of the LLM even improves pure-text scores, by 3.4 points on CMMLU [4].

Table 3.4: LLaVA-Prime ablation and comparison

Model	MME	MMBench	EgoSchema	Video-MME
LLaVA-1.5 (ViT-g + Qwen2-7B)	1569	70.6	45.0	55.1
+ Stage I video pretraining	1603	74.3	51.2	57.5
+ Stage II video pretraining	1621	73.5	52.4	58.8
+ Stage III with CHA	1610	73.6	54.3	60.2
LLaVA-OneVision-7B	1580	80.8	60.1	58.2
LLaVA-Prime-7B	1664	81.2	62.6	63.8

3.4.5 Discussion

LLaVA-Prime shows that video is a scalable and data-efficient source of visual knowledge that benefits image tasks as well as video tasks, and that CHA makes video affordable by keeping token counts at image level. Its limitations are the very large compute budget, which few groups can reproduce; a modest margin over strong baselines on some image benchmarks; and some inconsistencies in reporting, such as 26 versus 27 datasets for the zero-shot average. Code and models are promised but were not yet available at publication.

3.5 Summary of the Reviewed Papers

Table 3.5 summarises the four works; the next chapter compares them in detail.

Table 3.5: Summary of the reviewed works

Work	Problem	Technique	Main outcome
Mirage [1]	Text-only reasoning on visual tasks	Latent visual tokens; two-stage SFT; GRPO	Better spatial reasoning and planning without generating images
AURORA [2]	Weak depth and counting perception	Depth and box perception tokens; distillation; curriculum	Large gains on harder depth and counting benchmarks
ZipVL [3]	Slow, memory-hungry LVLM inference	Adaptive important-token ratio; sparse attention; KV eviction	Faster prefill and decoding with near-full accuracy
LLaVA-Prime [4]	Costly, static image-text data	Four-stage video pretraining; causal hierarchical aggregation	Stronger image and video understanding

Chapter 4

Discussion & Comparison

This chapter compares the four reviewed works. All build on the same vision-language architecture, but they target different weaknesses—reasoning, perception, efficiency, and training data—and intervene at different points of the pipeline. The comparison highlights their complementary strengths and the trade-offs among accuracy, cost, and generality.

4.1 Analytical Focus and Methodology

Table 4.1 compares the methodologies. Two works change *what the model outputs while it reasons*: Mirage adds continuous latent visual tokens [1] and AURORA adds discrete perception tokens [2]. ZipVL changes *how the model computes*, without any retraining [3], and LLaVA-Prime changes *what the model learns from*, through video pretraining [4].

Table 4.1: Methodological comparison of the reviewed works

Aspect	**Mirage [1]**	**AURORA [2]**	**ZipVL [3]**	**LLaVA-Prime [4]**
Target	Visual reasoning	Visual perception	Inference cost	Pretraining data
Base model	Qwen2.5-VL 7B, 3B	LLaVA 1.5 13B	LLaVA, LLaVA-Next, LongVA	ViT-g + Qwen2 7B, 72B
New element	Latent visual tokens	Depth and box tokens	Dynamic token sparsity	Video stages and CHA
Training	Two-stage SFT, GRPO	Distillation, curriculum	None	Four-stage pretraining
Inputs	Images	Images	Images, video	Images, video
Evaluation	VSP, Jigsaw, SAT, COMT	BLINK, CV-Bench, SEED	VQAv2, ChartQA, Video-MME	MME, MM-Bench, Video-MME

4.2 Visual Tokens for Reasoning: Latent versus Explicit

Mirage and AURORA share the idea that a VLM should reason with visual intermediate steps, but they make opposite design choices. AURORA's perception tokens are **explicit and discrete**: they decode into depth maps or boxes, which makes them interpretable and lets them be supervised by specialist models, but each depth map costs 100 tokens and the vocabulary must be extended for every new kind of intrinsic [2]. Mirage's latent tokens are **implicit and continuous**: only four vectors, anchored to the visual subspace in Stage 1 and then freed to serve the answer in Stage 2, so they are cheaper and more flexible but cannot be inspected directly [1]. Mirage reports that AURORA-style explicit generation performs poorly on interleaved spatial planning, suggesting that compact latent cues suit multi-step reasoning, while explicit tokens remain attractive when the intermediate result itself—a depth map or a set of boxes—is useful.

4.3 Efficiency and Scalability

The works sit at opposite ends of the cost spectrum. ZipVL *reduces* inference cost with no training, keeping roughly 30–50% of tokens and achieving up to 2.3 times faster prefill and 2.8 times higher decoding throughput [3]. LLaVA-Prime *increases* training cost substantially—hundreds of GPUs for weeks—but CHA keeps inference cost at image level by compressing video tokens [4]. Mirage and AURORA add modest inference cost, four latent tokens or about a hundred perception tokens, in exchange for better reasoning. These approaches are complementary: a model pretrained with video, reasoning with latent or perception tokens, could be served with dynamic token sparsity.

4.4 Training Data and Supervision

All four works depend on data engineering. Mirage synthesises helper images with tools and reasoning traces with a larger VLM [1]; AURORA generates pseudo-depth maps with Depth Anything and boxes from LVIS [2]; and LLaVA-Prime relies on 32M to 128M automatically captioned videos from Panda-70M [4]. Synthetic supervision is cheap and scalable but can be noisy—Mirage notes that video-generated helper images for SAT lack ground truth—and none of the works studies how errors in synthetic data

affect reliability.

4.5 Strengths and Limitations

Table 4.2 summarises the main strengths and limitations of each work. Every method improves one part of the VLM pipeline at a cost elsewhere: Mirage and AURORA add training effort and new tokens, ZipVL trades some accuracy on hard tasks for speed, and LLaVA-Prime trades a large training budget for broader visual knowledge.

Table 4.2: Strengths and limitations of the reviewed works

Work	**Strengths**	**Limitations**
Mirage [1]	No image decoder; only four latent tokens; transfers to smaller models; ablation-backed design	Relies on synthetic helper images and traces; small spatial benchmarks; latent tokens not interpretable
AURORA [2]	Interpretable depth and box tokens; no external tools at inference; extensible to new intrinsics	Two tasks on one base model; long token sequences; trails closed-source models on counting
ZipVL [3]	Training-free; works with FlashAttention; one signal drives prefill and decoding savings	Attention-only sparsity; probe overhead at short lengths; accuracy loss on hard tasks
LLaVA-Prime [4]	Scalable video data; benefits image and video tasks; token count kept at image level	Very large compute budget; modest gains on some benchmarks; reporting inconsistencies

4.6 Evaluation and Overall Insights

The evaluations differ in scope. Mirage and AURORA use small, targeted benchmarks of spatial reasoning, depth, and counting, with roughly a thousand training examples per task; ZipVL and LLaVA-Prime evaluate on broad image and video suites. Gains are therefore not directly comparable. Taken together, the works show a consistent pattern. **First**, giving VLMs visual intermediate representations—latent or explicit—improves tasks that language alone handles poorly. **Second**, much of a VLM's visual input is redundant, so adaptive sparsity can cut cost with little loss. **Third**, richer data such as video improves even static image understanding. **Fourth**, these advances are complementary, so combining them in one system is the natural next step.

4.7 Towards a Combined System

Because the four works act at different stages, they can in principle be combined, as sketched in Figure 4.7.1. A vision encoder and LLM pretrained on video with causal hierarchical aggregation would supply temporal and static visual knowledge [4]; during inference, dynamic token sparsity would keep only the important visual tokens and shrink the KV cache [3]; and while generating its answer, the model would insert perception tokens when an explicit depth map or set of boxes is useful [2] and latent visual tokens when it needs to imagine a step [1]. Whether these gains add up, and how sparsity interacts with latent reasoning, has not yet been tested and is a natural direction for future work.

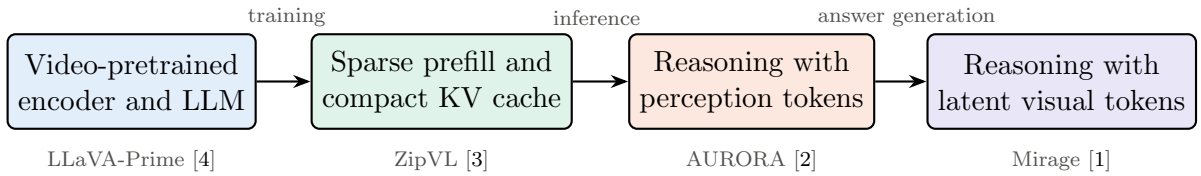


Figure 4.7.1: A combined pipeline using all four reviewed ideas

Chapter 5

Applications

5.1 Robotics and Embodied AI

Robots must plan paths, grasp objects, and judge distances. Latent visual reasoning improves spatial planning [1], perception tokens add depth awareness [2], and efficient inference makes on-board deployment feasible [3].

5.2 Autonomous Systems and Surveillance

Vehicles, drones, and cameras process long video streams in real time. Video-pretrained VLMs understand events and motion [4], while dynamic token sparsity reduces latency and memory for long inputs [3].

5.3 Healthcare and Assistive Technology

VLMs can describe medical images, answer clinicians' questions, and help visually impaired people understand their surroundings. Reliable counting and spatial reasoning [1, 2] are important for such safety-critical uses.

5.4 Document, Chart, and Education Tools

Chart and document understanding, which ZipVL evaluates with ChartQA and TextVQA [3], supports report analysis and tutoring, and step-by-step visual reasoning [1] helps explain answers to students.

Chapter 6

Challenges and Future Directions

6.1 Challenges Identified in the Reviewed Works

Mirage [1] depends on synthesised helper images and traces and is tested on four spatial benchmarks. **AURORA** [2] covers only depth and counting on one base model and trails closed-source models on counting. **ZipVL** [3] sparsifies only attention and still loses accuracy on hard tasks such as ChartQA. **LLaVA-Prime** [4] requires a very large compute budget that few groups can reproduce.

6.2 Cross-Cutting Challenges

The field faces **interpretability** limits, since latent tokens are hard to inspect; **hallucination**, where VLMs describe things that are not present; **data quality**, as synthetic supervision can be noisy; **evaluation gaps**, with small or inconsistent benchmarks; and **compute cost** for training and serving large models.

6.3 Future Research Directions

Promising directions are **unified systems** that combine video pretraining, latent or perception-token reasoning, and sparse inference; **interpretable latent reasoning** that can decode latent tokens when needed; **extending sparsity** to MLP layers [3]; **broader perception tokens** for segmentation, pose, and motion [2]; and **efficient video learning** with fewer GPUs [4].

Chapter 7

Conclusion

Multimodal AI connects computer vision with language-based understanding, and vision-language models have become its central tool. This seminar reviewed four recent works that address their main limitations. **Mirage** [1], the base paper, lets VLMs reason with compact latent visual tokens instead of generating images, improving spatial reasoning and planning. **AURORA** [2] adds depth and bounding-box perception tokens as intermediate reasoning steps, improving relative depth estimation and counting. **ZipVL** [3] selects important tokens dynamically in each layer, accelerating prefill and decoding with little loss of accuracy. **LLaVA-Prime** [4] pretrains VLMs on large-scale video with causal hierarchical aggregation, strengthening both image and video understanding.

The comparison shows that visual intermediate representations improve reasoning that language alone handles poorly, that adaptive sparsity removes much of the redundancy in visual input, and that video is a rich, scalable source of training data. Because these advances act at different points of the same pipeline, they are complementary. The field is therefore moving towards multimodal systems that perceive richly, reason visually, and run efficiently—combining latent reasoning, perception-aware learning, efficient computation, and better data to build more accurate, efficient, and reliable multimodal intelligence.

References

[1] Z. Yang, X. Yu, D. Chen, M. Shen, and C. Gan, "Machine mental imagery: Empower multimodal reasoning with latent visual tokens," in *Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition*, 2026, pp. 33510–33520.

[2] M. Bigverdi, Z. Luo, C.-Y. Hsieh, E. Shen, *et al.*, "Perception tokens enhance visual reasoning in multimodal language models," in *Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition*, 2025, pp. 3836–3845.

[3] Y. He, F. Chen, J. Liu, W. Shao, *et al.*, "ZipVL: Accelerating vision-language models through dynamic token sparsity," in *Proc. IEEE/CVF Int. Conf. Computer Vision*, 2025, pp. 20477–20486.

[4] Y. Zhang, H. Li, J. Liu, and X. Yue, "Learning beyond still frames: Scaling vision-language models with video," in *Proc. IEEE/CVF Int. Conf. Computer Vision*, 2025, pp. 22425–22435.

[5] A. Radford, J. W. Kim, C. Hallacy, A. Ramesh, G. Goh, S. Agarwal, *et al.*, "Learning transferable visual models from natural language supervision," in *Proc. Int. Conf. Machine Learning*, 2021, pp. 8748–8763.

[6] H. Liu, C. Li, Q. Wu, and Y. J. Lee, "Visual instruction tuning," in *Advances in Neural Information Processing Systems*, vol. 36, 2023.

[7] A. Dosovitskiy, L. Beyer, A. Kolesnikov, *et al.*, "An image is worth 16x16 words: Transformers for image recognition at scale," in *Proc. Int. Conf. Learning Representations*, 2021.

[8] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, *et al.*, "Attention is all you need," in *Advances in Neural Information Processing Systems*, vol. 30, 2017.

[9] J. Wei, X. Wang, D. Schuurmans, M. Bosma, *et al.*, "Chain-of-thought prompting elicits reasoning in large language models," in *Advances in Neural Information Processing Systems*, vol. 35, 2022, pp. 24824–24837.

[10] A. van den Oord, O. Vinyals, and K. Kavukcuoglu, "Neural discrete representation learning," in *Advances in Neural Information Processing Systems*, vol. 30, 2017.

[11] Z. Shao, P. Wang, Q. Zhu, R. Xu, *et al.*, "DeepSeekMath: Pushing the limits of mathematical reasoning in open language models," arXiv preprint arXiv:2402.03300, 2024.